

COLECOVISION TOOLCHAIN DASHBOARD · SEPTEMBER 2026

Different tools, different priorities.

Amy Studio, CVBasic, z88dk, ugBASIC, devkitSMS, PVColLib, libcv, and NewColeco can all create ColecoVision software. This dashboard compares evidence, not promises.

The tests use size-oriented native settings validated for these fixtures. Occupied size excludes cartridge padding. Smaller does not automatically mean faster or better.

REPRODUCIBLE EVIDENCE

Six samples, eight solutions.

Occupied ROM bytes	Amy Studio	NewColeco	libcv	PVColLib	devkitSMS	CVBasic	z88dk	ugBASIC
Hello World	238	795	1,001	1,106	1,507	1,466	3,687	5,245
Warrior bitmap	3,254	3,643	4,691	4,525	4,713	4,948	4,976	18,034
Controller monitor	595	932	812	1,194	1,430	1,695	4,016	5,887
Sprite metasprite	983	1,142	1,413	1,304	1,681	1,845	2,902	7,718
Gameplay state update	1,460	1,253	1,291	1,308	2,126	2,453	2,878	10,479
Tile animation	1,387	1,376	1,627	1,530	1,888	2,982	2,893	6,505

Occupied ROM bytes	Amy Studio	NewColeco	libcv	PVColLib	devkitSMS	CVBasic	z88dk	ugBASIC
Six-sample total	7,917	9,141	10,835	10,967	13,345	15,389	21,352	53,868

All 48 ROMs booted and passed their GearColeco checks. Warrior matches pixel for pixel; the metasprite, state update, and tile animation pass exact VRAM or state oracles. Amy Studio has fifteen verified direct-to-VRAM codecs. Results apply to these versions, settings, fixtures, and oracles; they are not universal rankings. Sources, assets, ROMs, occupied sizes, and hashes are included in one archive.

Download evidence ZIP

Read detailed results

SPECIALIZED GRAPHICS EXPERIMENTS

Useful evidence, outside the main ranking.

The evidence ZIP also contains the eight-tool PutPixel Motion fixture derived from sverx's SGLib_CV example. Seven implementations pass the motion, two-controller, and Name Table checks; ugBASIC is marked PARTIAL because native PLOT works while FIRE remains neutral in this fixture. A source-only pixel, line, and circle study based on Oscar Toledo's CVBasic example is included as unranked work in progress.

AMY SOURCE + GEARCOLECO

All six benchmark cases.

Hello World

```
text screen
backdrop sky blue
print centered at 11, "HELLO, WORLD!"
screen on
```



HELLO, WORLD!

The smallest benchmark establishes each toolchain's fixed runtime cost.

Warrior bitmap

```
picture WarriorPicture:
  pattern from "@project/warrior.pattern.zx0" codec zx0
  color from "@project/warrior.color.zx0" codec zx0
end picture

show picture WarriorPicture
```

Exomizer proof: the evidence archive contains this equivalent Amy source, both .exo assets, and the exact 3,319-byte ROM for direct testing.

```
picture WarriorExomizer:
  pattern from "@project/warrior.pattern.exo" codec exomizer
  color from "@project/warrior.color.exo" codec exomizer
end picture

show picture WarriorExomizer
```



Full 256x192 Graphics II output, verified pixel-for-pixel across all eight solutions.

Controller monitor

```
InputLoop:
  wait
  if keypad(1) <> 255 then
    backdrop magenta
  elseif joypad(1).fire then
    backdrop light red
  elseif joypad(1).up then
    backdrop light green
  else
```

```
    backdrop dark blue
end if
goto InputLoop
```

CONTROLLER TEST

MOVE OR PRESS A BUTTON

KEYPAD IS PURPLE

Runtime-tested for neutral input, direction, keypad, and fire-button changes.

Animated three-color metasprite

A metasprite is one visual actor assembled from multiple hardware sprites. This example overlaps three 16x16 sprites for white, yellow, and black layers.

```
u8 PlayerX = 120
u8 PlayerY = 88
u8 AnimationFrame = 0
```

```
sprites simple
set sprite count 3
```

```
MainLoop:
    wait
```

```
if joypad(1).left then PlayerX -= 1
if joypad(1).right then PlayerX += 1
AnimationFrame = 1 - AnimationFrame
set sprite 0 to PlayerY,PlayerX,0,15
set sprite 1 to PlayerY,PlayerX,4,11
set sprite 2 to PlayerY,PlayerX,8,1
update sprites
goto MainLoop
```

SPRITE METASPRITE



MOVE WITH JOYSTICK

GearColeco runtime result. Three overlapping sprites stay within the TMS9918 scanline limit; the archive contains all six animation masks and eight ROMs.

Gameplay state update

This deterministic six-actor workload measures state dispatch, record-array updates, timers, movement, collisions, score, and checksum rather than rendering speed.

```
state machine WorldBehavior:
  Running calls UpdateActors
  Bonus calls UpdateBonus
```

```
end state machine

ActorState Actors[6]
u16 Score = 0
u16 Checksum = 0

for Tick = 0 to 47
    dispatch WorldMode using WorldBehavior
    ' UpdateActors handles records, timers, movement and collisions.
next
```

STATE UPDATE BENCHMARK

ACTORS	06
DONE	
SCORE	00425
CHECK	01478

Exact final oracle: state 1, 13 collisions, score 425, checksum 1478. Complete source and all eight ROMs are in the evidence archive.

Graphics II tile animation

One shared pattern animates every star while six stationary 3x2 ships alternate tile frames through Name Table updates.

```

AnimationLoop:
  wait
  Tick += 1
  if Tick & 3 = 0 then
    StarFrame = (StarFrame + 1) % 3
    copy StarPatterns[StarFrame * 8] count 8 to vram.pattern + StarTile * 8
  end if
  if Tick & 7 = 0 then
    ShipFrame = 1 - ShipFrame
    for Ship = 0 to 5
      put ShipTiles[ShipFrame * 6] frame size 3,2 at ShipX[Ship],ShipY[Ship]
    next
  end if
  goto AnimationLoop

```



GearColeco observed both animation phases and verified all pattern, color, and Name Table bytes with no stray ship tiles.

GRAPHICS II COMPRESSION CORPUS

42 pictures, 15 codecs, 630 exact checks.

Every 12,288-byte Pattern + Color picture round-tripped exactly. Podiums rank first-use ROM bytes: payload plus one linked decoder. Gallery cards show the three smallest payloads and Pletter's rank when it is outside the top three.

Rank	Codec	Payload total	Decoder	First-use places (picture)		
				1st	2nd	3rd
1	Exomizer 2	161,374	226	21	12	2
2	ZX0	166,556	133	21	18	1
3	DAN2	166,923	212	0	0	7
4	DAN3	167,186	205	0	0	16
5	DAN1	167,240	205	0	3	4
6	MegaLZ	171,016	162	0	0	1
7	aPLib Compact	171,484	244	0	0	0
8	Pletter	171,751	212	0	0	0
9	ZX7	171,788	136	0	0	2
10	BitBuster	172,941	166	0	0	0
11	ZX1	176,562	127	0	4	5
12	ZX2	177,582	115	0	5	4
13	LZF	193,412	117	0	0	0
14	Nibble	214,555	115	0	0	0



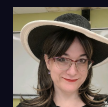
median

Cake

1. Exomizer 2: 2,935 bytes
2. ZX0: 2,958 bytes
3. DAN2: 2,999 bytes
- ... 9. Pletter: 3,108 bytes

Artwork by

Amy Bienvenu / NewColeco





least compressible

Mona Lisa

1. Exomizer 2: 9,208 bytes
2. ZX0: 9,426 bytes
3. DAN3: 9,490 bytes
- ... 5. Pletter: 9,640 bytes

Selection is mechanical. Exomizer uses a 226-byte ROM decoder plus 156 bytes of CPU RAM. Amy Studio reserves that workspace at a 256-byte boundary; its generated address varies and must not overlap other RAM. The detailed PDF contains all 42 converted images and per-picture evidence.

The compact philosophy predates Amy Studio.

NewColeco links Amy's historical CRTCV, CVLIB, GETPUT 1.1, and DAN2 code without ABI changes. Its Hello, bitmap, and controller ROMs remain small and run correctly today. The exact Warrior test confirms its direct-to-VRAM DAN2 path. Amy Studio continues that BIOS-aware, original-hardware approach with a modern language, integrated tools, and debugging.

PVCOLLIB EVIDENCE

A broad ColecoVision C library, not only a ROM-size entry.

Area	Verified local API or example	Status
Graphics	Text and bitmap modes, direct VRAM, sprites, hitbox collisions	Source verified; bitmap runtime exact
Sprite pressure	32-entry SAT and NMI sprite swapping for the four-per-line limit	Source verified
Controllers	Both joypads, keypads, four fire buttons, and both spinners	Source verified; visual ROM boots
Sound	BIOS-style sound tables, sequenced music, concurrent SFX areas	Source and examples verified
Compression	RLE, Pletter, DAN1, DAN2, and DAN3 direct-to-VRAM APIs	RLE exact; other stream pairings not ranked yet
Expanded hardware	MegaCart, SGM, Phoenix, and F18A support	Credited separately from stock tests

LIBCV EVIDENCE

Small linked C code, with a specialized bitmap codec.

Philipp Klaus Krause's libcv/libcvu completes all six runtime fixtures and ranks third at 10,835 occupied bytes. Its Warrior RLE+Huffman path is exact, but needs a 2,934-byte payload, 514-byte shared tree/configuration, about 411 decoder bytes, and 19 bytes of runtime RAM. Across 42 pictures it usually ranks 14th or 15th among 17 methods, so it informs a future compact shared-codebook experiment rather than replacing Amy's current codecs.

Stock and expanded hardware

The measured ranking targets an original ColecoVision with its TMS9918A, SN76489, 1 KB RAM, BIOS, and compact cartridge. Expanded hardware remains a valid strength, but is reported separately.

Extension	Examples of support	Comparison scope
MegaCart / banking	CVBasic, z88dk, devkitSMS, PVCoLLib	Valid, excluded from stock-ROM ranking
F18A video	PVCoLLib APIs and examples	Modified or compatible clone systems
SGM sound and RAM	CVBasic and PVCoLLib paths	Expansion or compatible clone systems

Extension	Examples of support	Comparison scope
ADAM / extra RAM	Machine-specific runtime work required	Future separate target, not stock ColecoVision

Philosophy

Area	Amy Studio	CVBasic	z88dk +coleco
Primary goal	Integrated ColecoVision game authoring	Portable BASIC game development	General-purpose Z80 C development
Target reach	ColecoVision-first	Multiple classic consoles and computers	Large family of Z80 targets
Typical workflow	Browser IDE, editors, compiler, debugger, ROM tests	Compiler plus editor and emulator	CLI toolchain plus editor and emulator
Hardware philosophy	Stock ColecoVision and compact cartridges	Portable abstractions with platform extensions	Low-level C control and broad libraries

Language and game systems

Capability	Amy Studio	CVBasic	z88dk +coleco
Numeric types	Explicit 8/16/32-bit integers, fixed-point, FP5, packed BCD	Game-oriented 8/16-bit integers	C integer families and software floating point

Capability	Amy Studio	CVBasic	z88dk +coleco
Data structures	1D/2D arrays, records, record arrays, RAM overlays	Arrays and BASIC data constructs	C arrays, structs, unions, and pointers
Functions	Typed parameters, returns, ref parameters, recursion	Procedures and BASIC function facilities	Full C functions and calling conventions
State management	Typed state machines, scenes, overlay-aware RAM	Conventional BASIC control flow	C control flow and manual architecture
Input	Joypad, keypad, spinner, pressed/released edges, controller-aware lowering	Joypad, keypad, spinner	C input APIs; advanced behavior is generally manual
Sprites	High-level sprites, collision helpers, opt-in flicker with stable ranges	High-level sprites and flicker support	TMS9918 sprite APIs and manual game logic
Sound	BIOS sound tables, Tiny Sound, DSound; visual music workflow still improving	Sound and integrated music commands	PSG libraries and external conversion workflows

Production workflow

Capability	Amy Studio	CVBasic	z88dk +coleco
Integrated browser IDE	Yes	No official integrated IDE	No official integrated IDE
Source debugging	Breakpoints, ASM stepping, memory/VRAM, rewind, profiling	External emulator workflow	External emulator/debugger workflow
Automated ROM tests	GearColeco-backed runtime tests across optimizer profiles	Project-specific external setup	Project-specific external setup

Capability	Amy Studio	CVBasic	z88dk +coleco
Graphics workflow	Integrated tile, sprite, animation, bitmap, and import tools	External/companion conversion tools	External tools and custom pipelines
Compression	Fifteen integrated direct-to-VRAM codecs, including Exomizer 2	Pletter-centered workflow	ZX-family tools and C/ASM libraries
ROM beyond 32 KB	Not a current goal	Banked ROM support	Banked/MegaCart options

AMY STUDIO FITS WHEN

You want depth on the ColecoVision.

You value original-hardware constraints, integrated graphics and compression, readable game-oriented syntax, explicit RAM accounting, source debugging, and an environment shaped by decades of ColecoVision practice.

Open Amy Studio

OTHER TOOLCHAINS FIT WHEN

Your project values their strengths.

CVBasic supports portable game-oriented BASIC. z88dk provides broad C and Z80 tooling. ugBASIC offers extensive cross-platform BASIC features. devkitSMS, PVCoLLib, and libcv provide established SDCC library workflows.

[CVBasic](#) · [z88dk](#) · [ugBASIC](#) · [devkitSMS](#) · [PVCoLLib](#) · [libcv](#)

Comparison maintained with the Amy Studio release

[Current Amy capabilities](#)

[Amy Studio source](#)